

STRAVYX

Australia's On-Demand Drone Services Network

DJI Bridge & Client Internals

MQTT ingestion | throttle | visibility projection | useMissionStream reconnect

Stravix Pty Ltd ACN 696 964 271 | Jul 2026 | CONFIDENTIAL

Source: docs/dji-bridge-and-client-internals.md. The implementation layer below the technical spec: how the NestJS Cloud API bridge ingests DJI MQTT, throttles, projects by role and fans out; and a reference useMissionStream client with snapshot + reconnect. Decision context: ADR 0001 (unified types, Option A). Illustrative code - harden, test and security-review before production.

1. Bridge Topology

Flow: DJI gateways (Pilot 2 / Dock) -> MQTT broker (EMQX) -> NestJS ingestion (normalize -> throttle -> project) -> Redis snapshot + Redis pub/sub -> WS gateway pods -> clients.

- Broker (EMQX) is the natural MQTT ingress; handles TLS/auth/QoS.
- Redis pub/sub decouples ingestion from WS pods - scale each independently, no sticky routing.
- Redis snapshot (last-known state per mission/device) hydrates newly-subscribed clients instantly - critical for reconnect.

2. MQTT Subscription

```
const TOPICS = [  
  'thing/product/+/osd',    // high-freq telemetry  
  'thing/product/+/state',  // sparse state changes  
  'thing/product/+/events', // upload progress, HMS  
  'sys/product/+/status',   // online/offline, update_topo  
];  
  
this.client = mqtt.connect(MQTT_URL, {  
  protocol: 'mqtts', reconnectPeriod: 2000,  
  clean: false,           // durable session: don't lose msgs on blip  
  clientId: `stravix-bridge-${POD_ID}`,  
});  
this.client.on('connect', () => this.client.subscribe(TOPICS, { qos: 1 }));
```

- clean:false + qos:1 = durable, at-least-once -> pipeline must be idempotent (dedupe on seq/event id).
- One broker connection per pod; broker shared-subscription (\$share/bridge/...) can load-balance OSD across pods.

3. Ingestion Pipeline: normalize -> throttle -> project -> publish

3.1 Normalization (raw thing-model -> packages/types)

```
private normalizeTelemetry(sn, missionId, osd): Telemetry {
  return {
    missionId, deviceSn: sn,
    position: { lat: osd.latitude, lng: osd.longitude, altM: osd.height },
    headingDeg: osd.attitude_head ?? osd.head,
    speedMs: Math.hypot(osd.horizontal_speed ?? 0, osd.vertical_speed ?? 0),
    batteryPct: osd.battery?.capacity_percent ?? osd.battery_percent,
    capturedAt: new Date().toISOString(), // bridge clock (device clock skews)
    seq: this.nextSeq(missionId), // monotonic per mission
  };
}
```

Normalization is the ONLY place DJI field names appear. Everything downstream is packages/types.

3.2 Throttling / coalescing

OSD arrives ~10 Hz; clients need ~1-2 Hz. Coalesce per mission with leading edge + trailing flush: keep the newest sample, emit at most every INTERVAL_MS (~700ms), always flush the last. state and HMS/upload events are NOT throttled (low-frequency, each matters); upload always emits terminal done/failed.

```
if (now - s.last >= INTERVAL_MS) { s.last = now; publish(t); } // leading edge
else {
  s.pending = t; // keep newest
  s.timer ??= setTimeout(() => { // trailing flush
    if (s.pending) { publish(s.pending); s.pending = undefined; }
  }, INTERVAL_MS - (now - s.last));
}
```

3.3 Visibility projection layer (the margin firewall)

Projection runs BEFORE publish, keyed by the SUBSCRIBER's role. Store full normalized object once (admin), project per role at fan-out. Publish to role-scoped Redis channels (mission:{id}:telemetry:{role}) so a WS pod subscribes only to its clients' roles - no client-side filtering, no leak path.

```
export function projectTelemetry(t: Telemetry) {
  const consumer: Partial<Telemetry> = { // safe subset only
    missionId: t.missionId, position: t.position,
    batteryPct: t.batteryPct, seq: t.seq, capturedAt: t.capturedAt,
  };
  return { admin: t, mobile_operator: t, consumer };
}
export function projectDeliverable(d, role) {
  if (d.layer === 2 && role === 'mobile_operator') return null; // operator NEVER sees L2
  return d; // no cost fields on type
}
```

3.4 Snapshot store + publish

```
await redis.set(`snap:mission:${id}:telemetry:${role}`, JSON.stringify(data), 'EX', 120);
await redis.publish(`mission:${id}:telemetry:${role}`,
  JSON.stringify({ type: 'telemetry', id, seq: t.seq, data }));
```

Snapshot keys hold the last-known projected value (short TTL); WS gateway reads them to build the initial snapshot frame.

4. WS Gateway (fan-out to clients)

```
async handleSubscribe(client, { id }) {
  await authz.assertCanView(client.userId, client.role, id); // RBAC on mission
  const snap = await buildSnapshot(id, client.role); // 1) hydrate from Redis
  client.emit({ type: 'snapshot', channel: 'mission', id, data: snap });
  const channels = [`mission:${id}:telemetry:${client.role}`, ...];
  redisSub.subscribe(channels, (c, msg) => client.emit(JSON.parse(msg))); // 2) deltas
  client.onClose(() => redisSub.unsubscribe(channels));
}
```

- Auth once on WS upgrade; authorize per subscribe (can this user view this mission?).
- Any pod serves any client - state lives in Redis, so no sticky sessions.
- Scale: add WS pods behind LB; Redis pub/sub fans out to all.

5. Reference useMissionStream (client)

Single multiplexed socket; snapshot-then-deltas; seq ordering; heartbeat; backoff reconnect. Web + RN.

```
private scheduleReconnect() {
  const jitter = Math.random() * 300;
  setTimeout(() => this.connect(), this.backoff + jitter);
  this.backoff = Math.min(this.backoff * 2, this.maxBackoff); // exponential, cap 30s
}
// on reconnect: resend all subscribe frames; server replies with fresh snapshot

// hook: drop out-of-order telemetry
case 'snapshot': lastSeq.current = f.data.telemetry?.seq ?? -1; hydrateAll(f.data); break;
case 'telemetry': if (f.seq <= lastSeq.current) return;
                  lastSeq.current = f.seq; setTelemetry(f.data); break;
case 'upload_progress': setUploads(u => ({ ...u, [f.data.fileId]: f.data })); break;
```

Behaviours delivered

- Snapshot-then-deltas: every (re)connect resends subscribe; server re-hydrates state in ~1 round-trip.
- Ordering: lastSeq guards out-of-order/duplicate telemetry (at-least-once upstream).
- Reconnect: exponential backoff + jitter, capped 30s; heartbeat detects dead sockets.
- Multiplexing: one socket app-wide; ref-counted subscriptions avoid duplicate streams per missionId.

6. Operational Concerns

Concern	Approach
Backpressure	Throttle at bridge; slow clients coalesce to snapshot cadence per key
Idempotency	Dedupe upstream events by id/seq; QoS 1 = at-least-once
Ordering across pods	seq per-mission monotonic (single writer per mission in bridge)
Security	JWT on upgrade + per-subscribe RBAC; role-scoped Redis channels
Scaling	Stateless WS pods + Redis pub/sub; broker shared-subscriptions for OSD
Observability	msgs/s per topic, throttle drop ratio, WS conns, reconnect rate, hydrate latency
Failure isolation	Ingestion crash != WS outage (separate services); snapshot TTL bounds staleness

7. Test Hooks

Target	Test
Normalizer	Replay recorded OSD/state/event fixtures -> assert exact packages/types output
Projection	Operator/consumer channel payloads contain no L2 / customerTotal keys
Throttle	10 Hz input -> assert <= ~2 Hz output and last sample always flushed
Reconnect	Kill socket mid-mission -> snapshot re-hydrates tracker < 2s; no stale pin
Ordering	Inject seq [5,4,6] -> client renders 5 then 6, ignores 4
RBAC	Subscribe to a mission the user can't view -> error frame, no data

Markdown source: docs/dji-bridge-and-client-internals.md. Related: dji-frontend-technical-spec.md, adr/0001-dji-telemetry-unified-types.md. Reference code is illustrative - harden and security-review before production.